面向国际留学生的信息管理系统V1.0源代码

1 源代码属性结构说明

1.1 项目技术栈

本软件采用 H5、Node.js、Express 和 MySQL 技术栈实现。前端由 HTML、CSS 和 JavaScript 组成，提供学生端双语 H5 页面以及教师端、管理端的横向管理界面。后端由 Node.js 和 Express 提供 REST API，包含登录认证、学生档案、教师管理、成绩适配、统计分析和风险查询等接口。数据库使用 MySQL，保存用户、学生、教师、班级、国籍、住宿、签证、学习记录、外部学校、成绩规则和成绩记录等业务数据。

1.2 源码根目录

源码根目录为当前项目目录。源码稿只选入与本系统运行直接相关的自研业务源码、完整后台代码、前端页面源码、数据库脚本、业务种子脚本和必要配置文件，不选入第三方依赖、锁文件、截图、PDF、缓存文件、说明书、Markdown 文档正文或软著材料生成脚本。

1.3 一级目录划分

- `src`：Node.js 后端源码，包含服务入口、路由、中间件、数据库连接和成绩适配服务。
- `public`：H5 前台和管理端静态资源，包含页面结构、样式和交互逻辑。
- `database`：MySQL 建库建表和基础数据脚本。
- `scripts`：软著材料生成脚本，不属于系统运行源码，本源码稿不纳入。
- `docs`：软著材料输出目录，不纳入源码正文。

1.4 二级与三级目录划分

```
.
├── database
│   └── schema.sql
├── public
│   ├── app.js
│   ├── favicon.svg
│   ├── index.html
│   └── styles.css
├── src
│   ├── config
│   │   └── db.js
│   ├── middleware
│   │   └── auth.js
│   ├── routes
│   │   ├── auth.js
│   │   ├── grades.js
│   │   ├── management.js
│   │   ├── students.js
│   │   └── teachers.js
│   ├── services
│   │   └── gradeAdapter.js
│   ├── initDb.js
│   ├── seedBusinessData.js
│   └── server.js
├── package.json
└── .env.example
```

1.5 主要模块归属关系

模块	文件范围	说明
用户认证	`src/routes/auth.js`、`src/middleware/auth.js`	登录、令牌签发、角色鉴权
学生档案	`src/routes/students.js`、`public/app.js`	本人档案、学生列表和学生端展示
教师管理	`src/routes/teachers.js`、`public/app.js`	教师列表、教师班级和教师端入口
成绩适配	`src/routes/grades.js`、`src/services/gradeAdapter.js`	多学校、多国家成绩换算、审核和异常记录

模块	文件范围	说明
风险查询	src/routes/management.js	签证、成绩、学习、住宿风险聚合
数据层	src/config/db.js、database/schema.sql、src/initDb.js、src/seedBusinessData.js	MySQL 连接、表结构、初始化和业务关联数据
前端界面	public/index.html、public/styles.css、public/app.js	H5 双语界面和管理端页面

1.6 选入源码稿范围说明

本源码稿按真实文件逐一输出，所有选入文件均来自当前软件工程并与系统运行直接相关。后台代码已完整纳入，包括服务入口、数据库连接、认证中间件、认证路由、学生路由、教师路由、成绩路由、统计与风险路由、成绩适配服务、数据库初始化脚本和业务数据种子脚本。配置文件和初始化脚本中涉及数据库密码、令牌密钥、演示密码、哈希、手机号、邮箱和外部地址的内容已脱敏，脱敏不改变文件结构和业务逻辑表达。

2 源码正文

2.1 文件：package.json

文件职责：项目依赖、启动命令和数据库脚本命令配置。

所属模块：项目配置。

```
0001  {
0002    "name": "international-student-information-platform",
0003    "version": "1.0.0",
0004    "description": "Information management and service platform for Southeast Asian international studen
ts.",
0005    "main": "src/server.js",
0006    "scripts": {
0007      "start": "node src/server.js",
0008      "dev": "nodemon src/server.js",
0009      "db:init": "node src/initDb.js",
0010      "db:seed": "node src/seedBusinessData.js"
0011    },
0012    "dependencies": {
0013      "bcryptjs": "^2.4.3",
0014      "cors": "^2.8.5",
0015      "dotenv": "^16.4.5",
0016      "express": "^4.19.2",
0017      "helmet": "^7.1.0",
0018      "jsonwebtoken": "^9.0.2",
0019      "mysql2": "^3.10.0"
0020    },
0021    "devDependencies": {
0022      "nodemon": "^3.1.4"
0023    }
0024  }
```

2.2 文件：.env.example

文件职责：环境变量示例配置，展示服务端口、数据库连接和令牌配置项。

所属模块：项目配置。

```
0001  PORT=3000
0002  DB_HOST=127.0.0.1
0003  DB_PORT=3306
0004  DB_USER=root
0005  DB_PASSWORD=***
0006  DB_NAME=international_student_platform
0007  JWT_SECRET=***
0008  JWT_EXPIRES_IN=8h
```

2.3 文件：src/server.js

文件职责：Node.js 后台服务入口，注册静态资源、接口路由和统一错误处理。

所属模块：后端服务入口。

```
const path = require('path');
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
require('dotenv').config();

const authRoutes = require('./routes/auth');
const studentRoutes = require('./routes/students');
const teacherRoutes = require('./routes/teachers');
const gradeRoutes = require('./routes/grades');
const managementRoutes = require('./routes/management');

const app = express();

app.use(helmet({ contentSecurityPolicy: false }));
app.use(cors());
app.use(express.json({ limit: '10mb' }));
app.get('/favicon.ico', (req, res) => {
  res.redirect(302, '/favicon.svg');
});
app.use(express.static(path.join(__dirname, '..', 'public')));

app.get('/api/health', (req, res) => {
  res.json({ status: 'ok', name: 'international-student-platform' });
});

app.use('/api/auth', authRoutes);
app.use('/api/students', studentRoutes);
app.use('/api/teachers', teacherRoutes);
app.use('/api/grades', gradeRoutes);
app.use('/api/management', managementRoutes);

app.use((req, res) => {
  res.status(404).json({ message: 'Not found' });
});

app.use((error, req, res, next) => {
  console.error(error);
  res.status(500).json({ message: 'Internal server error' });
});

if (require.main === module) {
  const port = Number(process.env.PORT || 3000);
  app.listen(port, () => {
    console.log(`Server running at http://localhost:${port}`);
  });
}

module.exports = app;
```

2.4 文件：`src/config/db.js`

文件职责：MySQL 连接池配置，提供后端数据访问基础能力。

所属模块：后端配置。

```
const mysql = require('mysql2/promise');
require('dotenv').config();

const pool = mysql.createPool({
  host: process.env.DB_HOST || '127.0.0.1',
  port: Number(process.env.DB_PORT || 3306),
  user: process.env.DB_USER || 'root',
  password: process.env.DB_PASSWORD || '***',
  database: process.env.DB_NAME || 'international_student_platform',
  waitForConnections: true,
  connectionLimit: 10,
  namedPlaceholders: true
});
```

```
0014
0015  module.exports = pool;
```

2.5 文件：src/middleware/auth.js

文件职责：JWT 鉴权中间件，按角色限制接口访问。

所属模块：认证与权限。

```
0001  const jwt = require('jsonwebtoken');
0002
0003  function auth(requiredRoles = []) {
0004    return (req, res, next) => {
0005      const header = req.headers.authorization || '';
0006      const token = header.startsWith('Bearer ') ? header.slice(7) : '';
0007
0008      if (!token) {
0009        return res.status(401).json({ message: 'Missing authorization token' });
0010      }
0011
0012      try {
0013        const payload = jwt.verify(token, process.env.JWT_SECRET || 'dev_secret');
0014        if (requiredRoles.length && !requiredRoles.includes(payload.role)) {
0015          return res.status(403).json({ message: 'Permission denied' });
0016        }
0017        req.user = payload;
0018        next();
0019      } catch (error) {
0020        return res.status(401).json({ message: 'Invalid or expired token' });
0021      }
0022    };
0023  }
0024
0025  module.exports = auth;
```

2.6 文件：src/routes/auth.js

文件职责：登录接口路由，完成账号校验、密码校验和令牌签发。

所属模块：认证与权限。

```
0001  const express = require('express');
0002  const bcrypt = require('bcryptjs');
0003  const jwt = require('jsonwebtoken');
0004  const pool = require('../config/db');
0005
0006  const router = express.Router();
0007
0008  router.post('/login', async (req, res, next) => {
0009    try {
0010      const { username, password } = req.body;
0011      const [rows] = await pool.execute(
0012        'SELECT id, username, password_hash, role, display_name, status FROM users WHERE username = :use
rname LIMIT 1',
0013        { username }
0014      );
0015
0016      const user = rows[0];
0017      if (!user || user.status !== 'active') {
0018        return res.status(401).json({ message: 'Invalid username or password' });
0019      }
0020
0021      const ok = await bcrypt.compare(password, user.password_hash);
0022      if (!ok) {
0023        return res.status(401).json({ message: 'Invalid username or password' });
0024      }
0025
0026      const token = jwt.sign(
0027        { id: user.id, username: user.username, role: user.role, name: user.display_name },
0028        process.env.JWT_SECRET || 'dev_secret',
0029        { expiresIn: process.env.JWT_EXPIRES_IN || '8h' }
0030      );
```

```
0031
0032      res.json({
0033        token,
0034        user: {
0035          id: user.id,
0036          username: user.username,
0037          role: user.role,
0038          displayName: user.display_name
0039        }
0040      });
0041    } catch (error) {
0042      next(error);
0043    }
0044  });
0045
0046  module.exports = router;
```

2.7 文件：`src/routes/students.js`

文件职责：学生档案接口路由，支持本人档案和学生列表查询。

所属模块：学生档案模块。

```
0001  const express = require('express');
0002  const pool = require('../config/db');
0003  const auth = require('../middleware/auth');
0004
0005  const router = express.Router();
0006
0007  router.get('/me', auth(['student']), async (req, res, next) => {
0008    try {
0009      const [rows] = await pool.execute(
0010        `SELECT s.*, c.name AS class_name, n.name_zh AS nationality_zh, n.name_en AS nationality_en
0011         FROM students s
0012         LEFT JOIN classes c ON c.id = s.class_id
0013         LEFT JOIN nationalities n ON n.id = s.nationality_id
0014         WHERE s.user_id = :userId`,
0015        { userId: req.user.id }
0016      );
0017      res.json(rows[0] || null);
0018    } catch (error) {
0019      next(error);
0020    }
0021  });
0022
0023  router.get('/', auth(['admin', 'teacher']), async (req, res, next) => {
0024    try {
0025      const [rows] = await pool.execute(
0026        `SELECT s.id, s.student_no, s.chinese_name, s.english_name, s.gender, s.enrollment_year,
0027                s.status, c.name AS class_name, n.name_zh AS nationality
0028         FROM students s
0029         LEFT JOIN classes c ON c.id = s.class_id
0030         LEFT JOIN nationalities n ON n.id = s.nationality_id
0031         ORDER BY s.id DESC`
0032      );
0033      res.json(rows);
0034    } catch (error) {
0035      next(error);
0036    }
0037  });
0038
0039  router.post('/', auth(['admin']), async (req, res, next) => {
0040    try {
0041      const data = req.body;
0042      const [result] = await pool.execute(
0043        `INSERT INTO students
0044         (user_id, student_no, chinese_name, english_name, gender, birth_date, nationality_id,
0045          passport_no, passport_expiry, phone, email, emergency_contact, enrollment_year,
0046          major, college, class_id, status)
0047         VALUES (:userId, :studentNo, :chineseName, :englishName, :gender, :birthDate, :nationalityId,
0048          :passportNo, :passportExpiry, :phone, :email, :emergencyContact, :enrollmentYear,
```

```
        :major, :college, :classId, :status)`,
      {
        userId: data.userId || null,
        studentNo: data.studentNo,
        chineseName: data.chineseName || null,
        englishName: data.englishName,
        gender: data.gender || null,
        birthDate: data.birthDate || null,
        nationalityId: data.nationalityId || null,
        passportNo: data.passportNo || null,
        passportExpiry: data.passportExpiry || null,
        phone: data.phone || null,
        email: data.email || null,
        emergencyContact: data.emergencyContact || null,
        enrollmentYear: data.enrollmentYear || null,
        major: data.major || null,
        college: data.college || null,
        classId: data.classId || null,
        status: data.status || 'active'
      }
    );
    res.status(201).json({ id: result.insertId });
  } catch (error) {
    next(error);
  }
});

module.exports = router;
```

2.8 文件：`src/routes/teachers.js`

文件职责：教师管理接口路由，支持教师列表和教师班级查询。

所属模块：教师管理模块。

```
const express = require('express');
const pool = require('../config/db');
const auth = require('../middleware/auth');

const router = express.Router();

router.get('/', auth(['admin']), async (req, res, next) => {
  try {
    const [rows] = await pool.execute(
      `SELECT t.*, u.username
       FROM teachers t
       LEFT JOIN users u ON u.id = t.user_id
       ORDER BY t.id DESC`
    );
    res.json(rows);
  } catch (error) {
    next(error);
  }
});

router.get('/me/classes', auth(['teacher']), async (req, res, next) => {
  try {
    const [rows] = await pool.execute(
      `SELECT c.*
       FROM classes c
       JOIN teachers t ON t.id = c.teacher_id
       WHERE t.user_id = :userId`,
      { userId: req.user.id }
    );
    res.json(rows);
  } catch (error) {
    next(error);
  }
});

router.post('/', auth(['admin']), async (req, res, next) => {
```

```
  try {
    const data = req.body;
    const [result] = await pool.execute(
      `INSERT INTO teachers (user_id, teacher_no, name, college, phone, email, teacher_type)
       VALUES (:userId, :teacherNo, :name, :college, :phone, :email, :teacherType)`,
      {
        userId: data.userId || null,
        teacherNo: data.teacherNo,
        name: data.name,
        college: data.college || null,
        phone: data.phone || null,
        email: data.email || null,
        teacherType: data.teacherType || 'course_teacher'
      }
    );
    res.status(201).json({ id: result.insertId });
  } catch (error) {
    next(error);
  }
});

module.exports = router;
```

2.9 文件：`src/routes/grades.js`

文件职责：成绩适配接口路由，支持成绩列表、适配、查询和审核。

所属模块：成绩适配模块。

```
const express = require('express');
const pool = require('../config/db');
const auth = require('../middleware/auth');
const { adaptGrade } = require('../services/gradeAdapter');

const router = express.Router();

async function findRule(schoolId, countryCode) {
  const [rows] = await pool.execute(
    `SELECT *
     FROM grade_rules
     WHERE (school_id = :schoolId OR school_id IS NULL)
       AND (country_code = :countryCode OR country_code IS NULL)
       AND status = 'active'
     ORDER BY school_id DESC, id DESC
     LIMIT 1`,
    { schoolId: schoolId || null, countryCode: countryCode || null }
  );
  const rule = rows[0];
  if (!rule) return {};

  return {
    gpaScale: rule.gpa_scale,
    creditRatio: rule.credit_ratio,
    passScore: rule.pass_score,
    version: rule.version
  };
}

router.get('/', auth(['admin', 'teacher']), async (req, res, next) => {
  try {
    const [rows] = await pool.execute(
      `SELECT gr.*, s.student_no, s.english_name, s.chinese_name, es.name AS source_school_name
       FROM grade_records gr
       JOIN students s ON s.id = gr.student_id
       LEFT JOIN external_schools es ON es.id = gr.source_school_id
       ORDER BY gr.id DESC
       LIMIT 200`
    );
    res.json(rows);
  } catch (error) {
    next(error);
```

```
0043    }
0044  });
0045
0046  router.post('/adapt', auth(['admin', 'teacher']), async (req, res, next) => {
0047    try {
0048      const input = req.body;
0049      const rule = await findRule(input.schoolId, input.countryCode);
0050      const adapted = adaptGrade(input, rule);
0051
0052      const [result] = await pool.execute(
0053        `INSERT INTO grade_records
0054         (student_id, course_code, course_name, source_school_id, source_country_code,
0055          original_grade, original_type, original_credit, adapted_score, adapted_gpa,
0056          adapted_credit, passed, rule_version, adapt_status, audit_status)
0057         VALUES (:studentId, :courseCode, :courseName, :schoolId, :countryCode,
0058          :originalGrade, :originalType, :originalCredit, :adaptedScore, :adaptedGpa,
0059          :adaptedCredit, :passed, :ruleVersion, 'success', 'pending')`,
0060        {
0061          studentId: input.studentId,
0062          courseCode: input.courseCode,
0063          courseName: input.courseName,
0064          schoolId: input.schoolId || null,
0065          countryCode: input.countryCode || null,
0066          originalGrade: String(input.originalGrade),
0067          originalType: input.originalType,
0068          originalCredit: input.originalCredit || 0,
0069          adaptedScore: adapted.adaptedScore,
0070          adaptedGpa: adapted.adaptedGpa,
0071          adaptedCredit: adapted.adaptedCredit,
0072          passed: adapted.passed ? 1 : 0,
0073          ruleVersion: adapted.ruleVersion
0074        }
0075      );
0076
0077      res.status(201).json({ id: result.insertId, ...adapted, auditStatus: 'pending' });
0078    } catch (error) {
0079      if (req.body && req.body.studentId) {
0080        await pool.execute(
0081          `INSERT INTO grade_records
0082           (student_id, course_code, course_name, source_school_id, source_country_code,
0083            original_grade, original_type, original_credit, adapt_status, exception_reason, audit_statu
s)
0084           VALUES (:studentId, :courseCode, :courseName, :schoolId, :countryCode,
0085            :originalGrade, :originalType, :originalCredit, 'exception', :reason, 'pending')`,
0086          {
0087            studentId: req.body.studentId,
0088            courseCode: req.body.courseCode || null,
0089            courseName: req.body.courseName || null,
0090            schoolId: req.body.schoolId || null,
0091            countryCode: req.body.countryCode || null,
0092            originalGrade: String(req.body.originalGrade || ''),
0093            originalType: req.body.originalType || null,
0094            originalCredit: req.body.originalCredit || 0,
0095            reason: error.message
0096          }
0097        );
0098      }
0099      res.status(422).json({ message: error.message });
0100    }
0101  });
0102
0103  router.get('/student/:studentId', auth(['admin', 'teacher', 'student']), async (req, res, next) => {
0104    try {
0105      if (req.user.role === 'student') {
0106        const [owned] = await pool.execute('SELECT id FROM students WHERE user_id = :userId', { userId:
req.user.id });
0107        if (!owned[0] || String(owned[0].id) !== String(req.params.studentId)) {
0108          return res.status(403).json({ message: 'Permission denied' });
0109        }
0110      }
0111
```

```
    const [rows] = await pool.execute(
      `SELECT gr.*, es.name AS source_school_name
       FROM grade_records gr
       LEFT JOIN external_schools es ON es.id = gr.source_school_id
       WHERE gr.student_id = :studentId
       ORDER BY gr.id DESC`,
      { studentId: req.params.studentId }
    );
    res.json(rows);
  } catch (error) {
    next(error);
  }
});

router.patch('/:id/audit', auth(['admin']), async (req, res, next) => {
  try {
    const { auditStatus } = req.body;
    await pool.execute(
      'UPDATE grade_records SET audit_status = :auditStatus, audited_by = :auditedBy, audited_at = NOW() WHERE id = :id',
      { auditStatus, auditedBy: req.user.id, id: req.params.id }
    );
    res.json({ id: Number(req.params.id), auditStatus });
  } catch (error) {
    next(error);
  }
});

module.exports = router;
```

2.10 文件：`src/routes/management.js`

文件职责：管理统计和风险查询接口路由。

所属模块：统计与风险模块。

```
const express = require('express');
const pool = require('../config/db');
const auth = require('../middleware/auth');

const router = express.Router();

router.get('/dashboard', auth(['admin', 'teacher']), async (req, res, next) => {
  try {
    const [[students]] = await pool.execute('SELECT COUNT(*) AS count FROM students');
    const [[teachers]] = await pool.execute('SELECT COUNT(*) AS count FROM teachers');
    const [[visaWarnings]] = await pool.execute(
      'SELECT COUNT(*) AS count FROM visas WHERE expiry_date BETWEEN CURDATE() AND DATE_ADD(CURDATE(), INTERVAL 60 DAY)'
    );
    const [[gradeExceptions]] = await pool.execute(
      "SELECT COUNT(*) AS count FROM grade_records WHERE adapt_status = 'exception'"
    );

    res.json({
      students: students.count,
      teachers: teachers.count,
      visaWarnings: visaWarnings.count,
      gradeExceptions: gradeExceptions.count
    });
  } catch (error) {
    next(error);
  }
});

router.get('/nationalities', auth(['admin', 'teacher', 'student']), async (req, res, next) => {
  try {
    const [rows] = await pool.execute('SELECT * FROM nationalities ORDER BY name_en');
    res.json(rows);
  } catch (error) {
    next(error);
```

```
0035    }
0036  });
0037
0038  router.get('/classes', auth(['admin', 'teacher']), async (req, res, next) => {
0039    try {
0040      const [rows] = await pool.execute(
0041        `SELECT c.*, t.name AS teacher_name
0042         FROM classes c
0043         LEFT JOIN teachers t ON t.id = c.teacher_id
0044         ORDER BY c.id DESC`
0045      );
0046      res.json(rows);
0047    } catch (error) {
0048      next(error);
0049    }
0050  });
0051
0052  router.get('/risks', auth(['admin', 'teacher']), async (req, res, next) => {
0053    try {
0054      const [visaRisks] = await pool.execute(
0055        `SELECT 'visa_expiring' AS risk_type, s.student_no, s.english_name, s.chinese_name,
0056                v.expiry_date AS risk_date,
0057                CONCAT('Visa expires on ', DATE_FORMAT(v.expiry_date, '%Y-%m-%d')) AS detail
0058         FROM visas v
0059         JOIN students s ON s.id = v.student_id
0060         WHERE v.expiry_date BETWEEN CURDATE() AND DATE_ADD(CURDATE(), INTERVAL 60 DAY)
0061         ORDER BY v.expiry_date ASC
0062         LIMIT 100`
0063      );
0064
0065      const [gradeRisks] = await pool.execute(
0066        `SELECT 'grade_exception' AS risk_type, s.student_no, s.english_name, s.chinese_name,
0067                gr.created_at AS risk_date,
0068                CONCAT(COALESCE(gr.course_name, 'Unknown course'), ': ', COALESCE(gr.exception_reason, g
r.adapt_status)) AS detail
0069         FROM grade_records gr
0070         JOIN students s ON s.id = gr.student_id
0071         WHERE gr.adapt_status = 'exception'
0072         ORDER BY gr.created_at DESC
0073         LIMIT 100`
0074      );
0075
0076      const [lowScoreRisks] = await pool.execute(
0077        `SELECT 'low_score' AS risk_type, s.student_no, s.english_name, s.chinese_name,
0078                gr.created_at AS risk_date,
0079                CONCAT(COALESCE(gr.course_name, 'Unknown course'), ': adapted score ', gr.adapted_score)
AS detail
0080         FROM grade_records gr
0081         JOIN students s ON s.id = gr.student_id
0082         WHERE gr.adapted_score IS NOT NULL AND gr.adapted_score < 60
0083         ORDER BY gr.adapted_score ASC
0084         LIMIT 100`
0085      );
0086
0087      const [learningRisks] = await pool.execute(
0088        `SELECT 'learning_warning' AS risk_type, s.student_no, s.english_name, s.chinese_name,
0089                sr.recorded_at AS risk_date,
0090                CONCAT(sr.course_name, ': ', sr.warning_level, ', attendance ', COALESCE(sr.attendance_r
ate, 0), '%') AS detail
0091         FROM study_records sr
0092         JOIN students s ON s.id = sr.student_id
0093         WHERE sr.warning_level IN ('warning', 'serious') OR sr.attendance_rate < 70
0094         ORDER BY sr.recorded_at DESC
0095         LIMIT 100`
0096      );
0097
0098      const [dormitoryRisks] = await pool.execute(
0099        `SELECT 'dormitory_exception' AS risk_type, s.student_no, s.english_name, s.chinese_name,
0100               d.check_out_date AS risk_date,
0101               CONCAT(d.building, '-', d.room_no, '-', d.bed_no, ': ', d.status) AS detail
0102        FROM dormitories d
```

```
        JOIN students s ON s.id = d.student_id
        WHERE d.status IN ('maintenance', 'reserved') OR d.check_out_date < CURDATE()
        ORDER BY d.check_out_date ASC
        LIMIT 100`
    );

    const items = [...visaRisks, ...gradeRisks, ...lowScoreRisks, ...learningRisks, ...dormitoryRisks];
    const summary = items.reduce((acc, item) => {
      acc[item.risk_type] = (acc[item.risk_type] || 0) + 1;
      return acc;
    }, {});

    res.json({ summary, items });
  } catch (error) {
    next(error);
  }
});

module.exports = router;
```

2.11 文件：src/services/gradeAdapter.js

文件职责：成绩适配核心服务，实现百分制、字母制、GPA 和通过制换算。

所属模块：成绩适配模块。

```
const LETTER_TO_PERCENT = {
  A: 95,
  'A-': 90,
  B: 85,
  'B-': 80,
  C: 75,
  'C-': 70,
  D: 65,
  F: 50
};

function gpaToPercent(value, scale) {
  const numeric = Number(value);
  const max = Number(scale || 4);
  if (Number.isNaN(numeric) || max <= 0) {
    throw new Error('Invalid GPA value');
  }
  return Math.round((numeric / max) * 100);
}

function toGpa4(percent) {
  if (percent >= 90) return 4.0;
  if (percent >= 85) return 3.7;
  if (percent >= 80) return 3.3;
  if (percent >= 75) return 3.0;
  if (percent >= 70) return 2.7;
  if (percent >= 65) return 2.3;
  if (percent >= 60) return 2.0;
  return 0;
}

function adaptGrade(input, rule = {}) {
  const type = String(input.originalType || '').toLowerCase();
  let percent;

  if (type === 'percent') {
    percent = Number(input.originalGrade);
  } else if (type === 'letter') {
    percent = LETTER_TO_PERCENT[String(input.originalGrade).trim().toUpperCase()];
  } else if (type === 'gpa') {
    percent = gpaToPercent(input.originalGrade, rule.gpaScale || input.gpaScale || 4);
  } else if (type === 'pass_fail') {
    percent = String(input.originalGrade).toLowerCase() === 'pass' ? 75 : 50;
  } else {
```

```
0045      throw new Error('Unsupported grade type');
0046    }
0047
0048    if (typeof percent !== 'number' || Number.isNaN(percent) || percent < 0 || percent > 100) {
0049      throw new Error('Adapted grade is out of range');
0050    }
0051
0052    const creditRatio = Number(rule.creditRatio || input.creditRatio || 1);
0053    return {
0054      adaptedScore: Math.round(percent),
0055      adaptedGpa: toGpa4(percent),
0056      adaptedCredit: Number((Number(input.originalCredit || 0) * creditRatio).toFixed(2)),
0057      passed: percent >= Number(rule.passScore || 60),
0058      ruleVersion: rule.version || 'default-v1'
0059    };
0060  }
0061
0062  module.exports = { adaptGrade };
```

2.12 文件：src/initDb.js

文件职责：数据库初始化脚本，执行建库建表 SQL。

所属模块：数据初始化。

```
0001  const fs = require('fs');
0002  const path = require('path');
0003  const mysql = require('mysql2/promise');
0004  require('dotenv').config();
0005
0006  async function initDb() {
0007    const sql = fs.readFileSync(path.join(__dirname, '..', 'database', 'schema.sql'), 'utf8');
0008    const connection = await mysql.createConnection({
0009      host: process.env.DB_HOST || '127.0.0.1',
0010      port: Number(process.env.DB_PORT || 3306),
0011      user: process.env.DB_USER || 'root',
0012      password: process.env.DB_PASSWORD || '***',
0013      multipleStatements: true
0014    });
0015
0016    await connection.query(sql);
0017    await connection.end();
0018    console.log('Database initialized.');
0019  }
0020
0021  initDb().catch((error) => {
0022    console.error(error.message);
0023    process.exit(1);
0024  });
```

2.13 文件：src/seedBusinessData.js

文件职责：业务关联测试数据种子脚本，生成学生、教师、班级、成绩和风险数据。

所属模块：数据初始化。

```
0001  const bcrypt = require('bcryptjs');
0002  const pool = require('./config/db');
0003  const { adaptGrade } = require('./services/gradeAdapter');
0004
0005  const passwordHash = bcrypt.hashSync('******', 10);
0006  const countryCodes = [
0007    'TH', 'VN', 'MY', 'ID', 'PH', 'SG', 'LA', 'KH', 'MM', 'BN',
0008    'CN', 'JP', 'KR', 'IN', 'PK', 'BD', 'LK', 'NP', 'MN', 'KZ'
0009  ];
0010  const gradeTypes = ['percent', 'letter', 'gpa', 'pass_fail'];
0011  const letters = ['A', 'A-', 'B', 'B-', 'C', 'C-', 'D', 'F'];
0012
0013  function pad(num, size = 3) {
0014    return String(num).padStart(size, '0');
0015  }
0016
```

```
0017  function datePlus(days) {
0018    const date = new Date();
0019    date.setDate(date.getDate() + days);
0020    return date.toISOString().slice(0, 10);
0021  }
0022
0023  async function execute(sql, params = {}) {
0024    await pool.execute(sql, params);
0025  }
0026
0027  async function seedUsers() {
0028    await execute(
0029      `INSERT INTO users (id, username, password_hash, role, display_name, status)
0030       VALUES (1, 'admin', :passwordHash, 'admin', 'System Admin', 'active')
0031       ON DUPLICATE KEY UPDATE password_hash = VALUES(password_hash), status = 'active'`,
0032      { passwordHash }
0033    );
0034
0035    for (let i = 1; i <= 120; i += 1) {
0036      await execute(
0037        `INSERT INTO users (id, username, password_hash, role, display_name, status)
0038         VALUES (:id, :username, :passwordHash, :role, :displayName, 'active')
0039         ON DUPLICATE KEY UPDATE password_hash = VALUES(password_hash), role = VALUES(role), display_nam
e = VALUES(display_name), status = 'active'`,
0040        {
0041          id: 1000 + i,
0042          username: `teacher${pad(i)}`,
0043          passwordHash,
0044          role: 'teacher',
0045          displayName: `Teacher ${pad(i)}`
0046        }
0047      );
0048    }
0049
0050    for (let i = 1; i <= 180; i += 1) {
0051      await execute(
0052        `INSERT INTO users (id, username, password_hash, role, display_name, status)
0053         VALUES (:id, :username, :passwordHash, 'student', :displayName, 'active')
0054         ON DUPLICATE KEY UPDATE password_hash = VALUES(password_hash), role = 'student', display_name =
VALUES(display_name), status = 'active'`,
0055        {
0056          id: 2000 + i,
0057          username: `student${pad(i)}`,
0058          passwordHash,
0059          displayName: `International Student ${pad(i)}`
0060        }
0061      );
0062    }
0063  }
0064
0065  async function seedNationalities() {
0066    for (let i = 1; i <= 120; i += 1) {
0067      const code = `${countryCodes[(i - 1) % countryCodes.length]}${pad(Math.ceil(i / countryCodes.lengt
h), 2)}`;
0068      await execute(
0069        `INSERT INTO nationalities (id, country_code, name_zh, name_en, continent, primary_language, tim
ezone)
0070         VALUES (:id, :countryCode, :nameZh, :nameEn, 'Asia-Pacific', :language, :timezone)
0071         ON DUPLICATE KEY UPDATE name_zh = VALUES(name_zh), name_en = VALUES(name_en), primary_language
= VALUES(primary_language), timezone = VALUES(timezone)`,
0072        {
0073          id: i,
0074          countryCode: code,
0075          nameZh: `国际生来源地${pad(i)}`,
0076          nameEn: `International Region ${pad(i)}`,
0077          language: `Language ${pad(((i - 1) % 20) + 1, 2)}`,
0078          timezone: `Asia/Test-${pad(((i - 1) % 24) + 1, 2)}`
0079        }
0080      );
0081    }
0082  }
```

```
0083
0084  async function seedTeachers() {
0085    for (let i = 1; i <= 120; i += 1) {
0086      await execute(
0087        `INSERT INTO teachers (id, user_id, teacher_no, name, college, phone, email, teacher_type)
0088         VALUES (:id, :userId, :teacherNo, :name, :college, :phone, :email, :teacherType)
0089         ON DUPLICATE KEY UPDATE user_id = VALUES(user_id), name = VALUES(name), college = VALUES(colleg
e), phone = VALUES(phone), email = VALUES(email), teacher_type = VALUES(teacher_type)`,
0090        {
0091          id: i,
0092          userId: 1000 + i,
0093          teacherNo: `T2026${pad(i)}`,
0094          name: `Teacher ${pad(i)}`,
0095          college: i % 2 === 0 ? 'International Education College' : 'School of Language Studies',
0096          phone: `1380000${pad(i, 4)}`,
0097          email: `teacher${pad(i)}@example.edu`,
0098          teacherType: ['course_teacher', 'head_teacher', 'counselor', 'admin_teacher'][i % 4]
0099        }
0100      );
0101    }
0102  }
0103
0104  async function seedClasses() {
0105    for (let i = 1; i <= 120; i += 1) {
0106      await execute(
0107        `INSERT INTO classes (id, code, name, college, major, teacher_id, enrollment_year)
0108         VALUES (:id, :code, :name, :college, :major, :teacherId, :year)
0109         ON DUPLICATE KEY UPDATE name = VALUES(name), college = VALUES(college), major = VALUES(major),
teacher_id = VALUES(teacher_id), enrollment_year = VALUES(enrollment_year)`,
0110        {
0111          id: i,
0112          code: `ISE-2026-${pad(i)}`,
0113          name: `International Student Class ${pad(i)}`,
0114          college: i % 2 === 0 ? 'International Education College' : 'School of Language Studies',
0115          major: ['Chinese Language', 'Business Chinese', 'Computer Science', 'International Trade'][i %
4],
0116          teacherId: ((i - 1) % 120) + 1,
0117          year: 2024 + (i % 3)
0118        }
0119      );
0120    }
0121  }
0122
0123  async function seedStudents() {
0124    for (let i = 1; i <= 180; i += 1) {
0125      await execute(
0126        `INSERT INTO students
0127         (id, user_id, student_no, chinese_name, english_name, gender, birth_date, nationality_id,
0128          passport_no, passport_expiry, phone, email, emergency_contact, enrollment_year, major, colleg
e, class_id, status)
0129         VALUES (:id, :userId, :studentNo, :chineseName, :englishName, :gender, :birthDate, :nationality
Id,
0130          :passportNo, :passportExpiry, :phone, :email, :emergencyContact, :enrollmentYear, :major, :col
lege, :classId, :status)
0131         ON DUPLICATE KEY UPDATE user_id = VALUES(user_id), chinese_name = VALUES(chinese_name), english
_name = VALUES(english_name),
0132          nationality_id = VALUES(nationality_id), class_id = VALUES(class_id), status = VALUES(status)
`,
0133        {
0134          id: i,
0135          userId: 2000 + i,
0136          studentNo: `S2026${pad(i)}`,
0137          chineseName: `留学生${pad(i)}`,
0138          englishName: `International Student ${pad(i)}`,
0139          gender: i % 2 === 0 ? 'Male' : 'Female',
0140          birthDate: `200${i % 6}-0${(i % 9) + 1}-15`,
0141          nationalityId: ((i - 1) % 120) + 1,
0142          passportNo: `P${pad(i, 6)}`,
0143          passportExpiry: datePlus(365 + i),
0144          phone: `1390000${pad(i, 4)}`,
0145          email: `student${pad(i)}@example.com`,
```

```
0146          emergencyContact: `Family Contact ${pad(i)}`,
0147          enrollmentYear: 2024 + (i % 3),
0148          major: ['Chinese Language', 'Business Chinese', 'Computer Science', 'International Trade'][i %
4],
0149          college: i % 2 === 0 ? 'International Education College' : 'School of Language Studies',
0150          classId: ((i - 1) % 120) + 1,
0151          status: ['active', 'active', 'active', 'delayed', 'suspended'][i % 5]
0152        }
0153      );
0154    }
0155  }
0156
0157  async function seedDormitories() {
0158    for (let i = 1; i <= 180; i += 1) {
0159      await execute(
0160        `INSERT INTO dormitories (id, building, room_no, bed_no, student_id, check_in_date, check_out_da
te, status)
0161         VALUES (:id, :building, :roomNo, :bedNo, :studentId, :checkIn, :checkOut, :status)
0162         ON DUPLICATE KEY UPDATE student_id = VALUES(student_id), check_out_date = VALUES(check_out_dat
e), status = VALUES(status)`,
0163        {
0164          id: i,
0165          building: `International Dorm ${((i - 1) % 8) + 1}`,
0166          roomNo: `${100 + ((i - 1) % 60)}`,
0167          bedNo: `${((i - 1) % 4) + 1}`,
0168          studentId: i,
0169          checkIn: '2026-09-01',
0170          checkOut: i % 17 === 0 ? datePlus(-5) : datePlus(200 + i),
0171          status: i % 29 === 0 ? 'maintenance' : 'occupied'
0172        }
0173      );
0174    }
0175  }
0176
0177  async function seedVisas() {
0178    for (let i = 1; i <= 180; i += 1) {
0179      await execute(
0180        `INSERT INTO visas (id, student_id, visa_type, visa_no, issue_date, expiry_date, residence_permi
t_no, document_url, status)
0181         VALUES (:id, :studentId, :visaType, :visaNo, :issueDate, :expiryDate, :permitNo, :documentUrl,
:status)
0182         ON DUPLICATE KEY UPDATE expiry_date = VALUES(expiry_date), status = VALUES(status)`,
0183        {
0184          id: i,
0185          studentId: i,
0186          visaType: i % 2 === 0 ? 'X1' : 'X2',
0187          visaNo: `VISA${pad(i, 6)}`,
0188          issueDate: '2026-01-10',
0189          expiryDate: i % 9 === 0 ? datePlus(20 + (i % 30)) : datePlus(180 + i),
0190          permitNo: `RP${pad(i, 6)}`,
0191          documentUrl: `/uploads/visa/${pad(i)}.pdf`,
0192          status: i % 9 === 0 ? 'expiring' : 'valid'
0193        }
0194      );
0195    }
0196  }
0197
0198  async function seedStudyRecords() {
0199    for (let i = 1; i <= 240; i += 1) {
0200      const attendance = 55 + (i % 46);
0201      await execute(
0202        `INSERT INTO study_records
0203         (id, student_id, course_code, course_name, attendance_rate, homework_status, class_performance,
warning_level, teacher_id)
0204         VALUES (:id, :studentId, :courseCode, :courseName, :attendance, :homeworkStatus, :performance,
:warningLevel, :teacherId)
0205         ON DUPLICATE KEY UPDATE attendance_rate = VALUES(attendance_rate), warning_level = VALUES(warni
ng_level), teacher_id = VALUES(teacher_id)`,
0206        {
0207          id: i,
0208          studentId: ((i - 1) % 180) + 1,
```

```
0209          courseCode: `CN${100 + (i % 12)}`,
0210          courseName: ['Chinese Listening', 'Chinese Speaking', 'Chinese Reading', 'Academic Writing'][i
% 4],
0211          attendance,
0212          homeworkStatus: attendance < 70 ? 'Incomplete' : 'Completed',
0213          performance: attendance < 70 ? 'Needs support' : 'Normal',
0214          warningLevel: attendance < 65 ? 'serious' : attendance < 75 ? 'warning' : 'none',
0215          teacherId: ((i - 1) % 120) + 1
0216        }
0217      );
0218    }
0219  }
0220
0221  async function seedExternalSchools() {
0222    for (let i = 1; i <= 120; i += 1) {
0223      await execute(
0224        `INSERT INTO external_schools (id, country_code, name, contact_name, contact_email, integration_
type, endpoint_url, status)
0225         VALUES (:id, :countryCode, :name, :contactName, :contactEmail, :integrationType, :endpointUrl,
'active')
0226         ON DUPLICATE KEY UPDATE country_code = VALUES(country_code), name = VALUES(name), integration_t
ype = VALUES(integration_type), endpoint_url = VALUES(endpoint_url), status = 'active'`,
0227        {
0228          id: i,
0229          countryCode: `${countryCodes[(i - 1) % countryCodes.length]}${pad(Math.ceil(i / countryCodes.l
ength), 2)}`,
0230          name: `Partner School ${pad(i)}`,
0231          contactName: `Partner Contact ${pad(i)}`,
0232          contactEmail: `partner${pad(i)}@school.example`,
0233          integrationType: ['api', 'excel', 'csv', 'manual', 'scheduled_sync'][i % 5],
0234          endpointUrl: `https://partner${pad(i)}.example.edu/api/grades`
0235        }
0236      );
0237    }
0238  }
0239
0240  async function seedGradeRules() {
0241    for (let i = 1; i <= 120; i += 1) {
0242      await execute(
0243        `INSERT INTO grade_rules (id, school_id, country_code, name, version, gpa_scale, credit_ratio, p
ass_score, status, effective_from, effective_to)
0244         VALUES (:id, :schoolId, :countryCode, :name, :version, :gpaScale, :creditRatio, :passScore, 'ac
tive', '2026-01-01', '2028-12-31')
0245         ON DUPLICATE KEY UPDATE gpa_scale = VALUES(gpa_scale), credit_ratio = VALUES(credit_ratio), pas
s_score = VALUES(pass_score), status = 'active'`,
0246        {
0247          id: i,
0248          schoolId: i,
0249          countryCode: `${countryCodes[(i - 1) % countryCodes.length]}${pad(Math.ceil(i / countryCodes.l
ength), 2)}`,
0250          name: `Grade Adaptation Rule ${pad(i)}`,
0251          version: `RULE-2026-${pad(i)}`,
0252          gpaScale: i % 3 === 0 ? 5 : 4,
0253          creditRatio: 0.8 + ((i % 5) * 0.1),
0254          passScore: i % 4 === 0 ? 65 : 60
0255        }
0256      );
0257    }
0258  }
0259
0260  function originalGrade(i, type) {
0261    if (type === 'percent') return String(45 + (i % 56));
0262    if (type === 'letter') return letters[i % letters.length];
0263    if (type === 'gpa') return String(((i % 41) / 10).toFixed(1));
0264    return i % 6 === 0 ? 'fail' : 'pass';
0265  }
0266
0267  async function seedGradeRecords() {
0268    for (let i = 1; i <= 360; i += 1) {
0269      const type = gradeTypes[i % gradeTypes.length];
0270      const studentId = ((i - 1) % 180) + 1;
```

```
0271      const schoolId = ((i - 1) % 120) + 1;
0272      const countryCode = `${countryCodes[(schoolId - 1) % countryCodes.length]}${pad(Math.ceil(schoolId
/ countryCodes.length), 2)}`;
0273      const grade = originalGrade(i, type);
0274      const rule = {
0275        gpaScale: schoolId % 3 === 0 ? 5 : 4,
0276        creditRatio: 0.8 + ((schoolId % 5) * 0.1),
0277        passScore: schoolId % 4 === 0 ? 65 : 60,
0278        version: `RULE-2026-${pad(schoolId)}`
0279      };
0280
0281      if (i % 37 === 0) {
0282        await execute(
0283          `INSERT INTO grade_records
0284           (id, student_id, course_code, course_name, source_school_id, source_country_code,
0285            original_grade, original_type, original_credit, adapt_status, exception_reason, audit_statu
s)
0286           VALUES (:id, :studentId, :courseCode, :courseName, :schoolId, :countryCode,
0287            'N/A', 'letter', :credit, 'exception', 'Source grade cannot be mapped', 'pending')
0288           ON DUPLICATE KEY UPDATE adapt_status = 'exception', exception_reason = VALUES(exception_reaso
n), audit_status = 'pending'`,
0289          {
0290            id: i,
0291            studentId,
0292            courseCode: `CR${100 + (i % 20)}`,
0293            courseName: ['Mathematics', 'Chinese Language', 'Computer Basics', 'Economics'][i % 4],
0294            schoolId,
0295            countryCode,
0296            credit: 1 + (i % 4)
0297          }
0298        );
0299        continue;
0300      }
0301
0302      const adapted = adaptGrade(
0303        { originalType: type, originalGrade: grade, originalCredit: 1 + (i % 4) },
0304        rule
0305      );
0306
0307      await execute(
0308        `INSERT INTO grade_records
0309         (id, student_id, course_code, course_name, source_school_id, source_country_code,
0310          original_grade, original_type, original_credit, adapted_score, adapted_gpa, adapted_credit,
0311          passed, rule_version, adapt_status, audit_status, audited_by, audited_at)
0312         VALUES (:id, :studentId, :courseCode, :courseName, :schoolId, :countryCode,
0313          :originalGrade, :originalType, :originalCredit, :adaptedScore, :adaptedGpa, :adaptedCredit,
0314          :passed, :ruleVersion, 'success', :auditStatus, 1, NOW())
0315         ON DUPLICATE KEY UPDATE adapted_score = VALUES(adapted_score), adapted_gpa = VALUES(adapted_gp
a),
0316          adapted_credit = VALUES(adapted_credit), passed = VALUES(passed), rule_version = VALUES(rule_v
ersion),
0317          adapt_status = 'success', audit_status = VALUES(audit_status)`,
0318        {
0319          id: i,
0320          studentId,
0321          courseCode: `CR${100 + (i % 20)}`,
0322          courseName: ['Mathematics', 'Chinese Language', 'Computer Basics', 'Economics'][i % 4],
0323          schoolId,
0324          countryCode,
0325          originalGrade: grade,
0326          originalType: type,
0327          originalCredit: 1 + (i % 4),
0328          adaptedScore: adapted.adaptedScore,
0329          adaptedGpa: adapted.adaptedGpa,
0330          adaptedCredit: adapted.adaptedCredit,
0331          passed: adapted.passed ? 1 : 0,
0332          ruleVersion: adapted.ruleVersion,
0333          auditStatus: i % 5 === 0 ? 'pending' : 'approved'
0334        }
0335      );
0336    }
```

```
0337  }
0338
0339  async function seedDemoAliases() {
0340    await execute(
0341      `INSERT INTO teachers (id, user_id, teacher_no, name, college, phone, email, teacher_type)
0342       VALUES (121, 2, 'T-DEMO-001', 'Teacher Demo', 'International Education College', '138********',
'***@example.com', 'head_teacher')
0343       ON DUPLICATE KEY UPDATE user_id = 2, name = VALUES(name), college = VALUES(college), phone = VALU
ES(phone), email = VALUES(email), teacher_type = VALUES(teacher_type)`
0344    );
0345
0346    await execute(
0347      `INSERT INTO classes (id, code, name, college, major, teacher_id, enrollment_year)
0348       VALUES (121, 'ISE-DEMO-001', 'Demo International Student Class', 'International Education Colleg
e', 'Chinese Language', 121, 2026)
0349       ON DUPLICATE KEY UPDATE name = VALUES(name), teacher_id = 121, enrollment_year = 2026`
0350    );
0351
0352    await execute(
0353      `INSERT INTO students
0354       (id, user_id, student_no, chinese_name, english_name, gender, birth_date, nationality_id,
0355        passport_no, passport_expiry, phone, email, emergency_contact, enrollment_year, major, college,
class_id, status)
0356       VALUES (181, 3, 'S-DEMO-001', '学生示例', 'Student Demo', 'Female', '2004-08-01', 1,
0357        'PDEMO001', :passportExpiry, '139********', '***@example.com', 'Family Demo', 2026,
0358        'Chinese Language', 'International Education College', 121, 'active')
0359       ON DUPLICATE KEY UPDATE user_id = 3, class_id = 121, status = 'active'`,
0360      { passportExpiry: datePlus(900) }
0361    );
0362
0363    await execute(
0364      `INSERT INTO dormitories (id, building, room_no, bed_no, student_id, check_in_date, check_out_dat
e, status)
0365       VALUES (181, 'International Dorm Demo', 'D101', '1', 181, '2026-09-01', :checkOut, 'occupied')
0366       ON DUPLICATE KEY UPDATE student_id = 181, check_out_date = VALUES(check_out_date), status = 'occu
pied'`,
0367      { checkOut: datePlus(360) }
0368    );
0369
0370    await execute(
0371      `INSERT INTO visas (id, student_id, visa_type, visa_no, issue_date, expiry_date, residence_permit_
no, document_url, status)
0372       VALUES (181, 181, 'X1', 'VISA-DEMO-001', '2026-01-10', :expiryDate, 'RP-DEMO-001', '/uploads/vis
a/demo.pdf', 'valid')
0373       ON DUPLICATE KEY UPDATE student_id = 181, expiry_date = VALUES(expiry_date), status = 'valid'`,
0374      { expiryDate: datePlus(360) }
0375    );
0376
0377    await execute(
0378      `INSERT INTO study_records
0379       (id, student_id, course_code, course_name, attendance_rate, homework_status, class_performance, w
arning_level, teacher_id)
0380       VALUES (241, 181, 'CN-DEMO', 'Demo Chinese Language', 92.50, 'Completed', 'Normal', 'none', 121)
0381       ON DUPLICATE KEY UPDATE student_id = 181, attendance_rate = VALUES(attendance_rate), warning_leve
l = 'none', teacher_id = 121`
0382    );
0383
0384    const adapted = adaptGrade(
0385      { originalType: 'letter', originalGrade: 'A', originalCredit: 3 },
0386      { creditRatio: 1, passScore: 60, version: 'DEMO-RULE-2026' }
0387    );
0388    await execute(
0389      `INSERT INTO grade_records
0390       (id, student_id, course_code, course_name, source_school_id, source_country_code,
0391        original_grade, original_type, original_credit, adapted_score, adapted_gpa, adapted_credit,
0392        passed, rule_version, adapt_status, audit_status, audited_by, audited_at)
0393       VALUES (500, 181, 'CN-DEMO', 'Demo Chinese Language', 1, 'TH01',
0394        'A', 'letter', 3, :adaptedScore, :adaptedGpa, :adaptedCredit,
0395        1, :ruleVersion, 'success', 'approved', 1, NOW())
0396       ON DUPLICATE KEY UPDATE adapted_score = VALUES(adapted_score), adapted_gpa = VALUES(adapted_gpa),
0397        adapted_credit = VALUES(adapted_credit), adapt_status = 'success', audit_status = 'approved'`,
```

```
    adapted
  );
}

async function main() {
  await seedUsers();
  await seedNationalities();
  await seedTeachers();
  await seedClasses();
  await seedStudents();
  await seedDormitories();
  await seedVisas();
  await seedStudyRecords();
  await seedExternalSchools();
  await seedGradeRules();
  await seedGradeRecords();
  await seedDemoAliases();
  await pool.end();
  console.log('Business seed data inserted.');
}

main().catch(async (error) => {
  console.error(error);
  await pool.end();
  process.exit(1);
});
```

2.14 文件：`database/schema.sql`

文件职责：MySQL 数据库建表脚本和基础数据脚本。

所属模块：数据层。

```
CREATE DATABASE IF NOT EXISTS international_student_platform
  DEFAULT CHARACTER SET utf8mb4
  DEFAULT COLLATE utf8mb4_unicode_ci;

USE international_student_platform;

CREATE TABLE IF NOT EXISTS users (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(64) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role ENUM('student', 'teacher', 'admin') NOT NULL,
  display_name VARCHAR(120) NOT NULL,
  status ENUM('active', 'disabled') NOT NULL DEFAULT 'active',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS nationalities (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  country_code VARCHAR(8) NOT NULL UNIQUE,
  name_zh VARCHAR(120) NOT NULL,
  name_en VARCHAR(120) NOT NULL,
  continent VARCHAR(60),
  primary_language VARCHAR(80),
  timezone VARCHAR(80)
);

CREATE TABLE IF NOT EXISTS teachers (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  user_id BIGINT UNIQUE,
  teacher_no VARCHAR(64) NOT NULL UNIQUE,
  name VARCHAR(120) NOT NULL,
  college VARCHAR(120),
  phone VARCHAR(40),
  email VARCHAR(120),
  teacher_type ENUM('course_teacher', 'head_teacher', 'counselor', 'admin_teacher') NOT NULL DEFAULT
'course_teacher',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_teachers_user FOREIGN KEY (user_id) REFERENCES users(id)
```

```
);

CREATE TABLE IF NOT EXISTS classes (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  code VARCHAR(64) NOT NULL UNIQUE,
  name VARCHAR(120) NOT NULL,
  college VARCHAR(120),
  major VARCHAR(120),
  teacher_id BIGINT,
  enrollment_year INT,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_classes_teacher FOREIGN KEY (teacher_id) REFERENCES teachers(id)
);

CREATE TABLE IF NOT EXISTS students (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  user_id BIGINT UNIQUE,
  student_no VARCHAR(64) NOT NULL UNIQUE,
  chinese_name VARCHAR(120),
  english_name VARCHAR(120) NOT NULL,
  gender VARCHAR(20),
  birth_date DATE,
  nationality_id BIGINT,
  passport_no VARCHAR(80),
  passport_expiry DATE,
  phone VARCHAR(40),
  email VARCHAR(120),
  emergency_contact VARCHAR(160),
  enrollment_year INT,
  major VARCHAR(120),
  college VARCHAR(120),
  class_id BIGINT,
  status ENUM('active', 'suspended', 'withdrawn', 'graduated', 'delayed') NOT NULL DEFAULT 'active',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_students_user FOREIGN KEY (user_id) REFERENCES users(id),
  CONSTRAINT fk_students_nationality FOREIGN KEY (nationality_id) REFERENCES nationalities(id),
  CONSTRAINT fk_students_class FOREIGN KEY (class_id) REFERENCES classes(id)
);

CREATE TABLE IF NOT EXISTS dormitories (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  building VARCHAR(120) NOT NULL,
  room_no VARCHAR(60) NOT NULL,
  bed_no VARCHAR(60) NOT NULL,
  student_id BIGINT,
  check_in_date DATE,
  check_out_date DATE,
  status ENUM('available', 'occupied', 'reserved', 'maintenance') NOT NULL DEFAULT 'available',
  CONSTRAINT fk_dormitories_student FOREIGN KEY (student_id) REFERENCES students(id)
);

CREATE TABLE IF NOT EXISTS visas (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  student_id BIGINT NOT NULL,
  visa_type VARCHAR(80) NOT NULL,
  visa_no VARCHAR(120),
  issue_date DATE,
  expiry_date DATE NOT NULL,
  residence_permit_no VARCHAR(120),
  document_url VARCHAR(255),
  status ENUM('valid', 'expiring', 'expired', 'renewing') NOT NULL DEFAULT 'valid',
  CONSTRAINT fk_visas_student FOREIGN KEY (student_id) REFERENCES students(id)
);

CREATE TABLE IF NOT EXISTS study_records (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  student_id BIGINT NOT NULL,
  course_code VARCHAR(80),
  course_name VARCHAR(160) NOT NULL,
  attendance_rate DECIMAL(5,2),
  homework_status VARCHAR(120),
```

```
  class_performance VARCHAR(255),
  warning_level ENUM('none', 'notice', 'warning', 'serious') NOT NULL DEFAULT 'none',
  teacher_id BIGINT,
  recorded_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_study_records_student FOREIGN KEY (student_id) REFERENCES students(id),
  CONSTRAINT fk_study_records_teacher FOREIGN KEY (teacher_id) REFERENCES teachers(id)
);

CREATE TABLE IF NOT EXISTS external_schools (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  country_code VARCHAR(8),
  name VARCHAR(180) NOT NULL,
  contact_name VARCHAR(120),
  contact_email VARCHAR(120),
  integration_type ENUM('api', 'excel', 'csv', 'manual', 'scheduled_sync') NOT NULL DEFAULT 'manual',
  endpoint_url VARCHAR(255),
  status ENUM('active', 'disabled') NOT NULL DEFAULT 'active'
);

CREATE TABLE IF NOT EXISTS grade_rules (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  school_id BIGINT,
  country_code VARCHAR(8),
  name VARCHAR(160) NOT NULL,
  version VARCHAR(40) NOT NULL,
  gpa_scale DECIMAL(4,2) NOT NULL DEFAULT 4.00,
  credit_ratio DECIMAL(6,3) NOT NULL DEFAULT 1.000,
  pass_score DECIMAL(5,2) NOT NULL DEFAULT 60.00,
  status ENUM('active', 'disabled') NOT NULL DEFAULT 'active',
  effective_from DATE,
  effective_to DATE,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_grade_rules_school FOREIGN KEY (school_id) REFERENCES external_schools(id)
);

CREATE TABLE IF NOT EXISTS grade_records (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  student_id BIGINT NOT NULL,
  course_code VARCHAR(80),
  course_name VARCHAR(160),
  source_school_id BIGINT,
  source_country_code VARCHAR(8),
  original_grade VARCHAR(40),
  original_type ENUM('percent', 'letter', 'gpa', 'pass_fail'),
  original_credit DECIMAL(6,2) DEFAULT 0,
  adapted_score DECIMAL(5,2),
  adapted_gpa DECIMAL(4,2),
  adapted_credit DECIMAL(6,2),
  passed TINYINT(1),
  rule_version VARCHAR(40),
  adapt_status ENUM('pending', 'success', 'exception') NOT NULL DEFAULT 'pending',
  exception_reason VARCHAR(255),
  audit_status ENUM('pending', 'approved', 'rejected', 'archived') NOT NULL DEFAULT 'pending',
  audited_by BIGINT,
  audited_at TIMESTAMP NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_grade_records_student FOREIGN KEY (student_id) REFERENCES students(id),
  CONSTRAINT fk_grade_records_school FOREIGN KEY (source_school_id) REFERENCES external_schools(id),
  CONSTRAINT fk_grade_records_auditor FOREIGN KEY (audited_by) REFERENCES users(id)
);

INSERT IGNORE INTO users (id, username, password_hash, role, display_name)
VALUES
  (1, 'admin', '***BCRYPT_HASH***', 'admin', 'System Admin'),
  (2, 'teacher01', '***BCRYPT_HASH***', 'teacher', 'Teacher Demo'),
  (3, 'student01', '***BCRYPT_HASH***', 'student', 'Student Demo');

UPDATE users
SET password_hash = '***BCRYPT_HASH***'
WHERE username IN ('admin', 'teacher01', 'student01');

```

```
INSERT IGNORE INTO nationalities (id, country_code, name_zh, name_en, continent, primary_language, timezone)
VALUES
  (1, 'TH', '泰国', 'Thailand', 'Asia', 'Thai', 'Asia/Bangkok'),
  (2, 'VN', '越南', 'Vietnam', 'Asia', 'Vietnamese', 'Asia/Ho_Chi_Minh'),
  (3, 'MY', '马来西亚', 'Malaysia', 'Asia', 'Malay', 'Asia/Kuala_Lumpur'),
  (4, 'ID', '印度尼西亚', 'Indonesia', 'Asia', 'Indonesian', 'Asia/Jakarta');

INSERT IGNORE INTO teachers (id, user_id, teacher_no, name, college, phone, email, teacher_type)
VALUES (1, 2, 'T2026001', 'Teacher Demo', 'International Education College', '138********', '***@example.com', 'head_teacher');

INSERT IGNORE INTO classes (id, code, name, college, major, teacher_id, enrollment_year)
VALUES (1, 'ISE-2026-A', 'International Student Class A', 'International Education College', 'Chinese Language', 1, 2026);

INSERT IGNORE INTO students
  (id, user_id, student_no, chinese_name, english_name, gender, birth_date, nationality_id,
   passport_no, passport_expiry, phone, email, emergency_contact, enrollment_year, major, college, class_id, status)
VALUES
  (1, 3, 'S2026001', '学生示例', 'Student Demo', 'Female', '2004-08-01', 1,
   'P000001', '2029-08-01', '139********', '***@example.com', 'Family +66 000000', 2026,
   'Chinese Language', 'International Education College', 1, 'active');

INSERT IGNORE INTO external_schools (id, country_code, name, integration_type, status)
VALUES
  (1, 'TH', 'Demo Partner University Thailand', 'api', 'active'),
  (2, 'VN', 'Demo Partner College Vietnam', 'excel', 'active');

INSERT IGNORE INTO grade_rules (id, school_id, country_code, name, version, gpa_scale, credit_ratio, pass_score, status)
VALUES
  (1, NULL, NULL, 'Default international grade adapter', 'default-v1', 4.00, 1.000, 60.00, 'active'),
  (2, 1, 'TH', 'Thailand partner GPA rule', 'TH-GPA-2026', 4.00, 1.000, 60.00, 'active');
```

2.15 文件：`public/index.html`

文件职责：H5 前台页面结构，包含登录、工作台、角色菜单和内容区域。

所属模块：前端页面。

```
<!doctype html>
<html lang="zh-CN">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>International Student Information Platform</title>
    <link rel="icon" href="/favicon.svg" type="image/svg+xml" />
    <link rel="stylesheet" href="/styles.css" />
  </head>
  <body>
    <header class="topbar">
      <div>
        <p class="eyebrow" data-i18n="systemTag">V1.0 留学生信息管理系统</p>
        <h1 data-i18n="systemName">面向国际留学生的信息管理系统</h1>
      </div>
      <div class="toolbar">
        <button class="icon-button" id="langToggle" title="Language">中 / EN</button>
        <button class="ghost-button hidden" id="logoutBtn" data-i18n="logout">退出</button>
      </div>
    </header>

    <main class="layout">
      <section class="login-panel" id="loginPanel">
        <div class="panel-title">
          <h2 data-i18n="loginTitle">用户登录</h2>
          <p data-i18n="loginHint">学生、教师和管理员使用统一入口登录。</p>
        </div>
        <form id="loginForm" class="form-grid">
          <label>
```

```
0030              <span data-i18n="username">账号</span>
0031              <input name="username" autocomplete="username" value="student01" required />
0032            </label>
0033            <label>
0034              <span data-i18n="password">密码</span>
0035              <input name="password" type="password" autocomplete="current-password" value="******" requ
ired />
0036            </label>
0037            <button class="primary-button" type="submit" data-i18n="loginButton">登录系统</button>
0038            <p class="message" id="loginMessage"></p>
0039          </form>
0040          <div class="demo-users">
0041            <span data-i18n="demoUsers">演示账号</span>
0042            <code>student01 / ******</code>
0043            <code>teacher01 / ******</code>
0044            <code>student001 / ******</code>
0045            <code>admin / ******</code>
0046          </div>
0047        </section>
0048
0049        <section class="workspace hidden" id="workspace">
0050          <nav class="tabs" id="tabs"></nav>
0051
0052          <section class="summary-grid" id="summaryGrid"></section>
0053
0054          <section class="content-grid">
0055            <article class="surface" id="primaryView"></article>
0056            <aside class="surface compact" id="sideView"></aside>
0057          </section>
0058        </section>
0059      </main>
0060
0061      <script src="/app.js"></script>
0062    </body>
0063  </html>
```

2.16 文件：`public/styles.css`

文件职责：H5 与管理端页面样式，包含移动端和桌面端布局。

所属模块：前端样式。

```
0001  :root {
0002    color-scheme: light;
0003    --bg: #f6f7f9;
0004    --surface: #ffffff;
0005    --surface-strong: #eef4f1;
0006    --line: #d8dee6;
0007    --text: #1f2933;
0008    --muted: #64707d;
0009    --accent: #1f7a6d;
0010    --accent-dark: #14594f;
0011    --warning: #b45f06;
0012    --danger: #b42318;
0013    --blue: #2f5f98;
0014    --shadow: 0 14px 34px rgba(21, 39, 60, 0.08);
0015  }
0016
0017  * {
0018    box-sizing: border-box;
0019  }
0020
0021  body {
0022    margin: 0;
0023    min-height: 100vh;
0024    background: var(--bg);
0025    color: var(--text);
0026    font-family: Arial, "Microsoft YaHei", sans-serif;
0027  }
0028
0029  button,
```

```
input,
select {
  font: inherit;
}

.topbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 24px;
  padding: 24px clamp(16px, 4vw, 48px);
  background: #ffffff;
  border-bottom: 1px solid var(--line);
}

.eyebrow {
  margin: 0 0 6px;
  color: var(--accent);
  font-size: 13px;
  font-weight: 700;
}

h1,
h2,
h3,
p {
  margin-top: 0;
}

h1 {
  margin-bottom: 0;
  font-size: clamp(22px, 3vw, 34px);
}

h2 {
  margin-bottom: 8px;
  font-size: 22px;
}

h3 {
  margin-bottom: 12px;
  font-size: 17px;
}

.toolbar,
.tabs,
.actions {
  display: flex;
  align-items: center;
  gap: 10px;
  flex-wrap: wrap;
}

.layout {
  width: min(1180px, calc(100% - 32px));
  margin: 28px auto 48px;
}

.login-panel,
.surface {
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: 8px;
  box-shadow: var(--shadow);
}

.login-panel {
  width: min(520px, 100%);
  padding: 26px;
}

```

```
.panel-title p,
.muted {
  color: var(--muted);
}

.form-grid {
  display: grid;
  gap: 16px;
}

label {
  display: grid;
  gap: 6px;
  color: var(--muted);
  font-size: 14px;
}

input,
select {
  width: 100%;
  min-height: 42px;
  padding: 9px 11px;
  border: 1px solid var(--line);
  border-radius: 6px;
  background: #ffffff;
  color: var(--text);
}

.primary-button,
.ghost-button,
.icon-button,
.tab-button {
  min-height: 40px;
  border-radius: 6px;
  border: 1px solid transparent;
  cursor: pointer;
}

.primary-button {
  background: var(--accent);
  color: #ffffff;
  font-weight: 700;
}

.primary-button:hover {
  background: var(--accent-dark);
}

.ghost-button,
.icon-button,
.tab-button {
  background: #ffffff;
  border-color: var(--line);
  color: var(--text);
}

.icon-button {
  min-width: 76px;
}

.tab-button.active {
  background: var(--surface-strong);
  border-color: var(--accent);
  color: var(--accent-dark);
  font-weight: 700;
}

.message {
  min-height: 20px;
  margin-bottom: 0;
  color: var(--danger);
```

```
}

.demo-users {
  display: grid;
  gap: 8px;
  margin-top: 18px;
  color: var(--muted);
  font-size: 13px;
}

code {
  display: inline-block;
  width: fit-content;
  padding: 4px 7px;
  background: #eef1f4;
  border-radius: 5px;
  color: #27313d;
}

.workspace {
  display: grid;
  gap: 18px;
}

.summary-grid {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 14px;
}

.metric {
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: 8px;
  padding: 18px;
}

.metric strong {
  display: block;
  margin-top: 8px;
  font-size: 28px;
}

.content-grid {
  display: grid;
  grid-template-columns: minmax(0, 2fr) minmax(280px, 0.9fr);
  gap: 18px;
}

.surface {
  padding: 20px;
}

.compact {
  align-self: start;
}

.data-table {
  width: 100%;
  border-collapse: collapse;
  font-size: 14px;
}

.data-table th,
.data-table td {
  padding: 11px 9px;
  border-bottom: 1px solid var(--line);
  text-align: left;
  vertical-align: top;
}

```

```
.data-table th {
  color: var(--muted);
  font-size: 13px;
}

.status {
  display: inline-flex;
  align-items: center;
  min-height: 24px;
  padding: 2px 8px;
  border-radius: 999px;
  background: #e7f3ef;
  color: var(--accent-dark);
  font-size: 12px;
  font-weight: 700;
}

.status.warning {
  background: #fff4df;
  color: var(--warning);
}

.status.danger {
  background: #fee4e2;
  color: var(--danger);
}

.profile-list {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 12px;
}

.profile-list div {
  padding: 12px;
  background: #f9fafb;
  border: 1px solid var(--line);
  border-radius: 6px;
}

.profile-list span {
  display: block;
  margin-bottom: 4px;
  color: var(--muted);
  font-size: 12px;
}

.hidden {
  display: none !important;
}

@media (max-width: 820px) {
  .topbar {
    align-items: flex-start;
    flex-direction: column;
  }

  .summary-grid,
  .content-grid,
  .profile-list {
    grid-template-columns: 1fr;
  }

  .login-panel {
    padding: 20px;
  }
}
```

2.17 文件：public/app.js

文件职责：前端交互逻辑，包含国际化、登录、菜单切换和接口调用。

所属模块：前端逻辑。

```
const i18n = {
  zh: {
    systemTag: 'V1.0 留学生信息管理系统',
    systemName: '面向国际留学生的信息管理系统',
    logout: '退出',
    loginTitle: '用户登录',
    loginHint: '学生、教师和管理员使用统一入口登录。',
    username: '账号',
    password: '密码',
    loginButton: '登录系统',
    demoUsers: '演示账号',
    profile: '个人档案',
    grades: '成绩适配',
    learning: '学习情况',
    teacher: '教师管理',
    students: '学生管理',
    dashboard: '统计分析',
    risks: '风险查询',
    source: '来源',
    status: '状态',
    adaptedScore: '适配成绩',
    adaptedGpa: '适配 GPA',
    auditStatus: '审核状态',
    noData: '暂无数据',
    studentCount: '学生人数',
    teacherCount: '教师人数',
    visaWarnings: '签证预警',
    gradeExceptions: '成绩异常',
    loginFailed: '登录失败，请检查账号或密码。',
    loadFailed: '数据加载失败，请确认后台和数据库已启动。',
    studentOnly: '学生端支持中英文切换并仅展示本人数据。',
    teacherScope: '教师端用于班级、课程、成绩录入与学习预警管理。',
    adminScope: '管理员端用于档案、国籍、班级、住宿、签证、教师、成绩规则和统计分析管理。'
  },
  en: {
    systemTag: 'V1.0 International Student Information System',
    systemName: 'Information Management System for International Students',
    logout: 'Logout',
    loginTitle: 'Sign In',
    loginHint: 'Students, teachers, and administrators use the same portal.',
    username: 'Username',
    password: 'Password',
    loginButton: 'Sign In',
    demoUsers: 'Demo accounts',
    profile: 'Profile',
    grades: 'Grade Adapter',
    learning: 'Learning',
    teacher: 'Teacher Management',
    students: 'Student Management',
    dashboard: 'Analytics',
    risks: 'Risk Query',
    source: 'Source',
    status: 'Status',
    adaptedScore: 'Adapted Score',
    adaptedGpa: 'Adapted GPA',
    auditStatus: 'Audit Status',
    noData: 'No data',
    studentCount: 'Students',
    teacherCount: 'Teachers',
    visaWarnings: 'Visa Alerts',
    gradeExceptions: 'Grade Exceptions',
    loginFailed: 'Sign-in failed. Check your username or password.',
    loadFailed: 'Failed to load data. Confirm the API and database are running.',
    studentOnly: 'The student client supports Chinese and English and only shows personal data.',
    teacherScope: 'The teacher portal supports classes, courses, grade entry, and learning alerts.',
    adminScope: 'The admin portal manages profiles, nationalities, classes, dormitories, visas, teache
rs, grade rules, and analytics.'
  }
};
```

```
0069
0070  let lang = localStorage.getItem('lang') || 'zh';
0071  let session = JSON.parse(localStorage.getItem('session') || 'null');
0072  let currentTab = 'profile';
0073
0074  const loginPanel = document.querySelector('#loginPanel');
0075  const workspace = document.querySelector('#workspace');
0076  const tabs = document.querySelector('#tabs');
0077  const summaryGrid = document.querySelector('#summaryGrid');
0078  const primaryView = document.querySelector('#primaryView');
0079  const sideView = document.querySelector('#sideView');
0080  const loginMessage = document.querySelector('#loginMessage');
0081  const logoutBtn = document.querySelector('#logoutBtn');
0082
0083  function t(key) {
0084    return i18n[lang][key] || key;
0085  }
0086
0087  function applyLanguage() {
0088    document.documentElement.lang = lang === 'zh' ? 'zh-CN' : 'en';
0089    document.querySelectorAll('[data-i18n]').forEach((node) => {
0090      node.textContent = t(node.dataset.i18n);
0091    });
0092    if (session) renderApp();
0093  }
0094
0095  async function api(path, options = {}) {
0096    const headers = {
0097      'Content-Type': 'application/json',
0098      ...(options.headers || {})
0099    };
0100    if (session?.token) headers.Authorization = `Bearer ${session.token}`;
0101    const response = await fetch(path, { ...options, headers });
0102    if (!response.ok) {
0103      const body = await response.json().catch(() => ({}));
0104      throw new Error(body.message || response.statusText);
0105    }
0106    return response.json();
0107  }
0108
0109  function setLoggedIn(value) {
0110    loginPanel.classList.toggle('hidden', value);
0111    workspace.classList.toggle('hidden', !value);
0112    logoutBtn.classList.toggle('hidden', !value);
0113  }
0114
0115  function roleTabs(role) {
0116    if (role === 'student') return ['profile', 'grades', 'learning'];
0117    if (role === 'teacher') return ['dashboard', 'students', 'grades', 'risks'];
0118    return ['dashboard', 'students', 'teacher', 'grades', 'risks'];
0119  }
0120
0121  function renderTabs() {
0122    tabs.innerHTML = roleTabs(session.user.role)
0123      .map((tab) => `<button class="tab-button ${tab === currentTab ? 'active' : ''}" data-tab="${tab}">
${t(tab)}</button>`)
0124      .join('');
0125    tabs.querySelectorAll('button').forEach((button) => {
0126      button.addEventListener('click', () => {
0127        currentTab = button.dataset.tab;
0128        renderApp();
0129      });
0130    });
0131  }
0132
0133  function metric(label, value, tone = '') {
0134    return `<div class="metric"><span class="muted">${label}</span><strong class="${tone}">${value}</str
ong></div>`;
0135  }
0136
0137  function table(headers, rows) {
```

```
0138    if (!rows.length) return `<p class="muted">${t('noData')}</p>`;
0139    return `
0140      <table class="data-table">
0141        <thead><tr>${headers.map((item) => `<th>${item}</th>`).join('')}</tr></thead>
0142        <tbody>${rows.join('')}</tbody>
0143      </table>
0144    `;
0145  }
0146
0147  async function renderDashboard() {
0148    const data = await api('/api/management/dashboard');
0149    summaryGrid.innerHTML = [
0150      metric(t('studentCount'), data.students),
0151      metric(t('teacherCount'), data.teachers),
0152      metric(t('visaWarnings'), data.visaWarnings, 'warning'),
0153      metric(t('gradeExceptions'), data.gradeExceptions, 'danger')
0154    ].join('');
0155    primaryView.innerHTML = `<h2>${t('dashboard')}</h2><p class="muted">${session.user.role === 'admin'
? t('adminScope') : t('teacherScope')}</p>`;
0156    sideView.innerHTML = `<h3>${t('status')}</h3><p class="status">${session.user.displayName}</p>`;
0157  }
0158
0159  async function renderStudentProfile() {
0160    const me = await api('/api/students/me');
0161    summaryGrid.innerHTML = [
0162      metric(t('profile'), me?.student_no || '-'),
0163      metric(lang === 'zh' ? '国籍' : 'Nationality', lang === 'zh' ? me?.nationality_zh || '-' : me?.nat
ionality_en || '-'),
0164      metric(lang === 'zh' ? '班级' : 'Class', me?.class_name || '-'),
0165      metric(t('status'), me?.status || '-')
0166    ].join('');
0167    primaryView.innerHTML = `
0168      <h2>${t('profile')}</h2>
0169      <div class="profile-list">
0170        <div><span>${lang === 'zh' ? '中文名' : 'Chinese Name'}</span>${me?.chinese_name || '-'}</div>
0171        <div><span>${lang === 'zh' ? 'English Name' : '英文名'}</span>${me?.english_name || '-'}</div>
0172        <div><span>${lang === 'zh' ? '护照号' : 'Passport No.'}</span>${me?.passport_no || '-'}</div>
0173        <div><span>${lang === 'zh' ? '护照有效期' : 'Passport Expiry'}</span>${me?.passport_expiry || '-
'}</div>
0174        <div><span>${lang === 'zh' ? '专业' : 'Major'}</span>${me?.major || '-'}</div>
0175        <div><span>${lang === 'zh' ? '邮箱' : 'Email'}</span>${me?.email || '-'}</div>
0176      </div>
0177    `;
0178    sideView.innerHTML = `<h3>${t('learning')}</h3><p class="muted">${t('studentOnly')}</p>`;
0179  }
0180
0181  async function renderStudentGrades(studentId) {
0182    const id = studentId || (await api('/api/students/me'))?.id;
0183    const grades = id ? await api(`/api/grades/student/${id}`) : [];
0184    summaryGrid.innerHTML = [
0185      metric(t('grades'), grades.length),
0186      metric(t('adaptedGpa'), grades[0]?.adapted_gpa || '-'),
0187      metric(t('auditStatus'), grades[0]?.audit_status || '-'),
0188      metric(t('status'), grades[0]?.adapt_status || '-')
0189    ].join('');
0190    primaryView.innerHTML = `
0191      <h2>${t('grades')}</h2>
0192      ${table(
0193        [lang === 'zh' ? '课程' : 'Course', t('source'), lang === 'zh' ? '原始成绩' : 'Original', t('adapt
edScore'), t('adaptedGpa'), t('auditStatus')],
0194        grades.map(
0195          (item) => `<tr>
0196            <td>${item.course_name || '-'}</td>
0197            <td>${item.source_school_name || item.source_country_code || '-'}</td>
0198            <td>${item.original_grade || '-'}</td>
0199            <td>${item.adapted_score || '-'}</td>
0200            <td>${item.adapted_gpa || '-'}</td>
0201            <td><span class="status">${item.audit_status}</span></td>
0202          </tr>`
0203        )
0204      )}
```

```
0205    `;
0206    sideView.innerHTML = `<h3>${lang === 'zh' ? '成绩适配接口要点' : 'Adapter Notes'}</h3><p class="mute
d">${lang === 'zh' ? '系统保留原始成绩、适配成绩、规则版本、审核状态和异常原因，支持跨国家、跨学校成绩归一化。' : 'The syst
em keeps original grades, adapted results, rule versions, audit states, and exception reasons for cross-coun
try grade normalization.'}</p>`;
0207  }
0208
0209  async function renderGradeManagement() {
0210    const grades = await api('/api/grades');
0211    summaryGrid.innerHTML = [
0212      metric(t('grades'), grades.length),
0213      metric(t('adaptedGpa'), grades[0]?.adapted_gpa || '-'),
0214      metric(t('auditStatus'), grades[0]?.audit_status || '-'),
0215      metric(lang === 'zh' ? '异常记录' : 'Exceptions', grades.filter((item) => item.adapt_status === 'ex
ception').length, 'danger')
0216    ].join('');
0217    primaryView.innerHTML = `
0218      <h2>${t('grades')}</h2>
0219      ${table(
0220        [lang === 'zh' ? '学号' : 'Student No.', lang === 'zh' ? '学生' : 'Student', lang === 'zh' ? '课
程' : 'Course', t('source'), lang === 'zh' ? '原始成绩' : 'Original', t('adaptedScore'), t('auditStatus')],
0221        grades.map(
0222          (item) => `<tr>
0223            <td>${item.student_no}</td>
0224            <td>${item.chinese_name || item.english_name}</td>
0225            <td>${item.course_name || '-'}</td>
0226            <td>${item.source_school_name || item.source_country_code || '-'}</td>
0227            <td>${item.original_grade || '-'}</td>
0228            <td>${item.adapted_score || '-'}</td>
0229            <td><span class="status ${item.adapt_status === 'exception' ? 'danger' : ''}">${item.audit_s
tatus}</span></td>
0230          </tr>`
0231        )
0232      )}
0233    `;
0234    sideView.innerHTML = `<h3>${lang === 'zh' ? '成绩适配管理' : 'Grade Adaptation'}</h3><p class="muted">
${lang === 'zh' ? '教师和管理员可查看外部学校成绩、原始成绩类型、适配分数、GPA、审核状态和异常记录，支撑跨国家、跨学校成绩统
一归档。' : 'Teachers and administrators can review external school grades, original types, adapted scores, G
PA, audit states, and exception records for unified archiving.'}</p>`;
0235  }
0236
0237  async function renderStudents() {
0238    const students = await api('/api/students');
0239    summaryGrid.innerHTML = [metric(t('studentCount'), students.length), metric(t('status'), session.use
r.role), metric(t('grades'), '-'), metric(t('visaWarnings'), '-')].join('');
0240    primaryView.innerHTML = `
0241      <h2>${t('students')}</h2>
0242      ${table(
0243        [lang === 'zh' ? '学号' : 'Student No.', lang === 'zh' ? '姓名' : 'Name', lang === 'zh' ? '国籍' :
'Nationality', lang === 'zh' ? '班级' : 'Class', t('status')],
0244        students.map(
0245          (item) => `<tr>
0246            <td>${item.student_no}</td>
0247            <td>${item.chinese_name || item.english_name}</td>
0248            <td>${item.nationality || '-'}</td>
0249            <td>${item.class_name || '-'}</td>
0250            <td><span class="status">${item.status}</span></td>
0251          </tr>`
0252        )
0253      )}
0254    `;
0255    sideView.innerHTML = `<h3>${t('teacher')}</h3><p class="muted">${session.user.role === 'teacher' ? t
('teacherScope') : t('adminScope')}</p>`;
0256  }
0257
0258  async function renderTeachers() {
0259    const teachers = await api('/api/teachers');
0260    summaryGrid.innerHTML = [metric(t('teacherCount'), teachers.length), metric(t('status'), 'active'),
metric(lang === 'zh' ? '权限' : 'Role', 'RBAC'), metric(lang === 'zh' ? '日志' : 'Logs', 'on')].join('');
0261    primaryView.innerHTML = `
0262      <h2>${t('teacher')}</h2>
```

```
      ${table(
        [lang === 'zh' ? '工号' : 'Teacher No.', lang === 'zh' ? '姓名' : 'Name', lang === 'zh' ? '学院' : 'College', lang === 'zh' ? '类型' : 'Type', lang === 'zh' ? '邮箱' : 'Email'],
        teachers.map(
          (item) => `<tr>
            <td>${item.teacher_no}</td>
            <td>${item.name}</td>
            <td>${item.college || '-'}</td>
            <td>${item.teacher_type}</td>
            <td>${item.email || '-'}</td>
          </tr>`
        )
      )}
    `;
    sideView.innerHTML = `<h3>${lang === 'zh' ? '教师端权限' : 'Teacher Permissions'}</h3><p class="muted">${t('teacherScope')}</p>`;
  }

  async function renderRisks() {
    const data = await api('/api/management/risks');
    const summary = data.summary || {};
    summaryGrid.innerHTML = [
      metric(lang === 'zh' ? '签证临期' : 'Visa Expiring', summary.visa_expiring || 0, 'warning'),
      metric(lang === 'zh' ? '成绩异常' : 'Grade Exceptions', summary.grade_exception || 0, 'danger'),
      metric(lang === 'zh' ? '低分风险' : 'Low Scores', summary.low_score || 0, 'danger'),
      metric(lang === 'zh' ? '学习预警' : 'Learning Alerts', summary.learning_warning || 0, 'warning')
    ].join('');
    primaryView.innerHTML = `
      <h2>${t('risks')}</h2>
      ${table(
        [lang === 'zh' ? '类型' : 'Type', lang === 'zh' ? '学号' : 'Student No.', lang === 'zh' ? '姓名' : 'Name', lang === 'zh' ? '日期' : 'Date', lang === 'zh' ? '详情' : 'Detail'],
        (data.items || []).map(
          (item) => `<tr>
            <td><span class="status ${item.risk_type.includes('exception') || item.risk_type === 'low_score' ? 'danger' : 'warning'}">${item.risk_type}</span></td>
            <td>${item.student_no}</td>
            <td>${item.chinese_name || item.english_name}</td>
            <td>${String(item.risk_date || '-').slice(0, 10)}</td>
            <td>${item.detail || '-'}</td>
          </tr>`
        )
      )}
    `;
    sideView.innerHTML = `<h3>${t('risks')}</h3><p class="muted">${lang === 'zh' ? '风险查询聚合签证、成绩、学习和住宿等关键业务风险，便于教师和管理员及时处理。' : 'Risk query aggregates visa, grade, learning, and dormitory risks for teachers and administrators.'}</p>`;
  }

  function renderLearningPlaceholder() {
    summaryGrid.innerHTML = [metric(t('learning'), '-'), metric(lang === 'zh' ? '考勤' : 'Attendance', '-'), metric(lang === 'zh' ? '预警' : 'Alerts', '-'), metric(lang === 'zh' ? '课程' : 'Courses', '-')].join('');
    primaryView.innerHTML = `<h2>${t('learning')}</h2><p class="muted">${lang === 'zh' ? '学习情况模块用于展示课程、考勤、作业、课堂表现、学习预警和教师评价。' : 'The learning module shows courses, attendance, assignments, classroom performance, alerts, and teacher evaluations.'}</p>`;
    sideView.innerHTML = `<h3>${t('status')}</h3><p class="status">${lang === 'zh' ? '待接入数据' : 'Awaiting records'}</p>`;
  }

  async function renderApp() {
    if (!session) return;
    setLoggedIn(true);
    renderTabs();
    try {
      if (currentTab === 'dashboard') await renderDashboard();
      if (currentTab === 'profile') await renderStudentProfile();
      if (currentTab === 'grades') {
        if (session.user.role === 'student') await renderStudentGrades();
        else await renderGradeManagement();
      }
```

```
0323      if (currentTab === 'students') await renderStudents();
0324      if (currentTab === 'teacher') await renderTeachers();
0325      if (currentTab === 'risks') await renderRisks();
0326      if (currentTab === 'learning') renderLearningPlaceholder();
0327    } catch (error) {
0328      primaryView.innerHTML = `<p class="message">${t('loadFailed')}</p><code>${error.message}</code>`;
0329      sideView.innerHTML = '';
0330    }
0331  }
0332
0333  document.querySelector('#langToggle').addEventListener('click', () => {
0334    lang = lang === 'zh' ? 'en' : 'zh';
0335    localStorage.setItem('lang', lang);
0336    applyLanguage();
0337  });
0338
0339  document.querySelector('#loginForm').addEventListener('submit', async (event) => {
0340    event.preventDefault();
0341    loginMessage.textContent = '';
0342    const form = new FormData(event.currentTarget);
0343    try {
0344      session = await api('/api/auth/login', {
0345        method: 'POST',
0346        body: JSON.stringify({
0347          username: form.get('username'),
0348          password: form.get('password')
0349        })
0350      });
0351      localStorage.setItem('session', JSON.stringify(session));
0352      currentTab = roleTabs(session.user.role)[0];
0353      renderApp();
0354    } catch (error) {
0355      loginMessage.textContent = t('loginFailed');
0356    }
0357  });
0358
0359  logoutBtn.addEventListener('click', () => {
0360    session = null;
0361    localStorage.removeItem('session');
0362    setLoggedIn(false);
0363  });
0364
0365  applyLanguage();
0366  if (session) {
0367    currentTab = roleTabs(session.user.role)[0];
0368    renderApp();
0369  } else {
0370    setLoggedIn(false);
0371  }
```

2.18 文件：`public/favicon.svg`

文件职责：系统图标资源，属于自研轻量界面资源。

所属模块：前端资源。

```
0001  <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64">
0002    <rect width="64" height="64" rx="12" fill="#1f7a6d"/>
0003    <path d="M16 18h32v6H16zM16 29h32v6H16zM16 40h22v6H16z" fill="#fff"/>
0004  </svg>
```